# Swap System Integration Overview

The swap subsystem, page aging mechanism, and out-of-memory (OOM) handling together form the foundation of a demand-paging system. Each component performs a specific role, and when integrated, they would allow the kernel to handle memory pressure gracefully — swapping out cold pages instead of immediately killing processes.

## 1. Memory Pressure and the OOM Trigger

The starting point of this process is **memory pressure**, which occurs when `page_alloc()` fails to return a free page.
Currently, the kernel's reaction is to call `oom_kill_task()`, which scans all active user tasks, finds the one with the largest resident set size (RSS), and terminates it to reclaim memory.

With the swap system in place, this point becomes the natural place to attempt page reclamation through swapping before resorting to killing any process.
The call sequence would look like this:

page_alloc() → allocation fails

↓

try_swap_out_pages()

↓

if still no memory → oom_kill_task()

## 2. Page Tracking and Aging

Each allocated page now contains metadata in its `page_info` structure:

- `virt_addr`: the virtual address where the page is mapped
- `owner`: pointer to the owning task
- `active_node`: used to link the page into global tracking lists

Whenever a page is allocated for a task inside `populate_region()`, these fields are initialized and the page is inserted into the **global active list (`active_pages`)**.

A background kernel thread, `page_check_daemon`, continuously scans this list:

1. It walks through all pages in the active list.
2. For each page, it finds the corresponding PTE in the owner's page table.
3. It checks and clears the **accessed bit**.
4. If the bit was not set (meaning the page hasn't been used recently), it moves the page from the **active list** to the **inactive list**.

This creates a separation between:

- **Active pages:** recently used, should stay in memory.
- **Inactive pages:** cold and eligible for swapping.

## 3. Swapping Pages Out

When the kernel decides to reclaim memory, it can now scan the inactive list to find pages that are safe to evict.
Each page selected for eviction goes through the following process:

1. **Locate a free swap slot.**
   The swap subsystem maintains a bitmap or table to track free and used swap slots.
2. **Write the page to the swap disk.**
   The `swap_page_out()` function computes the disk offset (LBA) for that slot and uses `disk_write()` to save the page contents to disk.
3. **Update the page table entry.**
   After a successful write, the PTE for that virtual address has its `PAGE_PRESENT` bit cleared and a custom `PAGE_PAGED_OUT` flag set.
4. **Free the physical frame.**
   The corresponding physical page is freed back to the buddy allocator, making memory available for new allocations.

Each swapped-out page is now represented in memory only by its swap entry (mapping PID, virtual address, and slot index), allowing the kernel to later retrieve it when needed.

## 4. Handling Page Faults and Swapping Pages In

If a process later accesses a page that has been swapped out, a **page fault** occurs. The fault handler detects that the PTE has the `PAGE_PAGED_OUT` flag set but not `PAGE_PRESENT`, indicating that the page's data resides on the swap disk.

In this case:

1. The handler allocates a new page frame using `page_alloc()`.
2. It looks up the corresponding swap entry to find the correct swap slot.
3. It calls `swap_page_in()`, which reads the page data from disk into the newly allocated page.
4. The PTE is updated to point to the new physical page with `PAGE_PRESENT` set.
5. The swap slot is marked as free again, allowing it to be reused later.

After this, the process continues as if the page had always been in memory.

## 5. Full System Workflow

Bringing all components together, the complete lifecycle would look like this:

1. **Page Allocation**
    - `populate_region()` allocates a page, fills in `virt_addr` and `owner`, and adds it to the active list.
2. **Page Aging**
    - `page_check_daemon` periodically scans active pages.
    - Recently accessed pages stay active; cold ones are moved to inactive.
3. **Memory Pressure Detected**
    - `page_alloc()` fails, indicating low memory.
    - The kernel invokes a swap-out routine before calling `oom_kill_task()`.

4. **Swapping Out**
    - Inactive pages are written to disk via `swap_page_out()`.
    - Their PTEs are marked as paged out, and their frames are freed.
5. **Page Faults on Swapped-Out Pages**
    - Access to a swapped-out page triggers a fault.
    - The handler detects `PAGE_PAGED_OUT` and calls `swap_page_in()` to restore it.
6. **Fallback: OOM Killing**
    - If swapping still cannot free enough memory, `oom_kill_task()` is executed to terminate the process with the highest RSS.

---

## 6. Summary

In summary, the swap and page-aging infrastructure transforms the kernel from a simple "allocate-or-kill" model into a more sophisticated virtual memory manager.
Instead of immediately terminating tasks when memory runs out, the kernel can now identify unused pages, write them to secondary storage, and later reload them on demand.

The combination of:

- Active/inactive page tracking,
- Swap-out and swap-in mechanisms, and
- The OOM killer as a final fallback

creates a complete and resilient memory management pipeline.

Due to time constraints it was not possible to debug and make all the parts work together. However, the code does logically work.